

NeuroScan AI

Complete Project Explanation and Demo Guide

Generated on 10 May 2026, 23:11

1. Project Introduction

NeuroScan AI is a full-stack AI/ML web application for brain tumour detection from MRI images. It combines a Streamlit frontend, authentication, SQLite database storage, image preprocessing, deep learning prediction, Grad-CAM explainability, segmentation overlay, PDF reporting, and dashboard analytics in one platform.

The project is designed as an educational and research-grade demonstration of how artificial intelligence can be integrated into a medical imaging workflow.

2. Problem Statement

Brain tumours can be life-threatening and require timely analysis. MRI scans are widely used for tumour identification, but manual interpretation takes expert time. The goal of this project is to assist MRI-based screening by building a system that can accept MRI uploads, classify the scan, explain the model focus area, highlight suspicious regions, and generate a report.

3. Main Objectives

- Build a secure web interface for MRI upload and analysis
- Classify MRI scans into Glioma, Meningioma, Pituitary, or Normal
- Preprocess images to create model-ready input
- Provide explainability with Grad-CAM
- Highlight suspicious tumour regions using segmentation overlay
- Generate PDF reports and save prediction history
- Store user, scan, and analytics data in SQLite
- Support future multi-model training and model comparison

4. Technologies Used

- Python for the core implementation
- Streamlit for the user interface
- TensorFlow / Keras for deep learning
- OpenCV for preprocessing and segmentation logic
- NumPy and Pandas for data handling
- Plotly, Matplotlib, and Seaborn for analytics and charts
- SQLite for database storage
- bcrypt for password hashing
- Report generation utilities for downloadable scan documents

5. Current Project Structure

- app.py: main Streamlit application and page router
- frontend/: UI theming and presentation helpers
- backend/: scan orchestration service
- authentication/: registration, login, OTP, session logic
- database/: SQLite schema and CRUD functions
- preprocessing/: image validation, preprocessing, dataset management
- model/: app-facing inference layer
- models/: training registry, evaluation, comparison pipeline

explainability/: Grad-CAM generation
segmentation/: tumour highlighting overlay
reports/: PDF-related generation scripts
utils/: shared helpers for paths, logging, summaries, and confidence interpretation

6. Dataset Details

The extracted MRI archive contains 7,200 images in total. The observed structure is Training and Testing folders with four categories.

Training/glioma: 1400
Training/meningioma: 1400
Training/notumor: 1400
Training/pituitary: 1400
Testing/glioma: 400
Testing/meningioma: 400
Testing/notumor: 400
Testing/pituitary: 400

7. User Workflow

The project follows a realistic end-user workflow. This is important to explain in the demo because it shows the system is more than a standalone ML notebook.

User registers and verifies the account with OTP
User logs in securely
User uploads an MRI image
The image is validated and previewed
The MRI is preprocessed into model input form
The deep learning model predicts the class
Grad-CAM and segmentation visuals are generated
Confidence, severity, quality, and AI summary are shown
Prediction is stored in SQLite
User downloads a report

8. Authentication Module Explanation

The authentication layer improves realism and security. Registration collects username, email, phone, and password. Passwords are hashed using bcrypt. OTP records are generated and verified to simulate phone-based validation. Streamlit session state tracks logged-in users, which restricts sensitive pages like upload, dashboard, and history.

9. Database Design

SQLite is used because it is lightweight, portable, and easy for a demo project. The database schema has been expanded beyond the original project.

users table stores account details and hashed passwords
otp_records stores OTP values, expiry, and usage state
patient_records stores optional patient metadata
predictions stores file details, class result, probabilities, severity, quality, summaries, and model info
model_versions stores best-model metadata for version tracking
analytics_events stores structured scan events for future analytics

10. Image Preprocessing Pipeline

Image preprocessing is necessary because uploaded MRI images can differ in shape, scale, and format. The pipeline standardizes all inputs before inference.

- Validate file type and size

- Convert to RGB if needed

- Resize to 224 x 224

- Apply Gaussian blur for mild denoising

- Normalize pixel values into the 0 to 1 range

- Add batch dimension for Keras input

- Estimate MRI quality using contrast and sharpness heuristics

11. Deep Learning Model Explanation

The app-facing prediction layer loads the current best model for inference. The training design supports transfer learning and multiple backbone families such as VGG16, ResNet50, EfficientNetB0, and MobileNetV2.

- Transfer learning uses pretrained ImageNet features

- A classification head converts extracted features into four tumour classes

- Softmax output returns probabilities for all classes

- The highest probability class becomes the final prediction

12. Predicted Classes

- Glioma: glial-cell tumour, generally treated as a high-risk class

- Meningioma: tumour related to meninges, often benign but clinically relevant

- Pituitary: tumour in the pituitary region, often linked to endocrine effects

- Normal: no tumour pattern detected by the model

13. Grad-CAM Explainability

Grad-CAM helps explain why the model predicted a class. Instead of producing only a label, it highlights the image regions that influenced the model most strongly. This makes the system more interpretable and demo-friendly.

- Feature maps from the last convolutional layer are extracted

- Gradients are computed for the predicted class

- A heatmap is generated from weighted activations

- The heatmap is overlaid on the MRI

- The app derives a coarse focus-region description from the heatmap center

14. Segmentation Feature

The segmentation module uses OpenCV heuristics instead of a U-Net for now. It is still useful in a demo because it highlights suspicious bright regions and outlines them with contours.

- MRI is converted to grayscale

- Adaptive thresholding is applied

- Morphological operations clean the mask

- Contours are extracted

- Small noise regions are ignored

- A binary mask and overlay image are generated

15. AI Summary and Confidence Interpretation

The project converts raw prediction outputs into human-readable explanation. Confidence values are mapped to bands like Very high, High, Moderate, or Low confidence. Severity is estimated from the predicted class and confidence. The system also generates a concise AI summary and assistant-style recommendation text.

16. Dashboard Analytics

The dashboard adds a full-stack analytics layer to the project.

- Total scans
- Tumour versus normal counts
- Prediction distribution charts
- MRI quality breakdown
- Recent scan history
- Model usage information

17. PDF Reporting Feature

The reporting flow packages the scan output into a downloadable document so the application behaves more like a practical AI platform.

- User and scan details
- Prediction and confidence
- Severity and model information
- Probability distribution
- Original MRI and preprocessed MRI
- Grad-CAM image
- Segmentation overlay
- AI summary and recommendations

18. Training Pipeline Design

The upgraded training pipeline is designed to support automatic dataset preparation, corrupted file handling, image organization, train/validation/test splitting, transfer learning, model comparison, and best-model selection.

- ZIP extraction
- Image validation
- Folder normalization
- Train/validation/test splits
- Data augmentation
- EarlyStopping
- ReduceLROnPlateau
- Checkpoint saving
- TensorBoard and CSV logs
- Best model chosen by evaluation metrics such as F1 score

19. Strengths of the Project

- Combines AI and web development in one application
- Provides explainability and segmentation, not just classification
- Includes secure user workflow and stored history
- Uses modular architecture for easier maintenance
- Demonstrates a realistic medical-AI platform flow

20. Limitations

- This is an educational and research project, not a clinical diagnostic system

Segmentation is heuristic and not a medical-grade segmentation model
Model performance depends on training quality and dataset characteristics
Hospital deployment would require stronger security, validation, and compliance

21. Future Enhancements

Integrate true U-Net segmentation
Support DICOM images
Add doctor/admin role-based access
Deploy with Docker and cloud infrastructure
Add REST APIs
Extend to multi-slice or 3D MRI analysis

22. Demo Presentation Flow

A simple way to present the project is: problem statement, objectives, technology stack, live application workflow, prediction result, Grad-CAM and segmentation output, dashboard analytics, architecture, and finally future scope.

23. Common Viva Questions

Why did you use Streamlit? Because it allows quick AI dashboard development in Python.
Why 224 x 224? Because it matches many pretrained CNN backbones.
Why transfer learning? It improves performance and reduces training time.
What is Grad-CAM? It is a visual explanation method for CNN predictions.
Why SQLite? It is lightweight and easy for demo deployment.
How are passwords stored? Using bcrypt hashing, not plain text.
What is the limitation? The project is educational and not a final clinical tool.

24. Conclusion

NeuroScan AI is a complete educational AI platform for brain MRI analysis. It is strong for demo and viva because it combines machine learning, image preprocessing, explainability, segmentation, authentication, database management, analytics, and reporting in one coherent system.